অধ্যায় ১৪

Advanced Functions, Closures, Higher-Order Patterns এবং Functional Programming

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি lexical scope ও closure ব্যাখ্যা করতে, callback ও higher-order function ডিজাইন করতে, `this` binding বুঝতে, function composition ও currying প্রয়োগ করতে, এবং immutable data transformation দিয়ে একটি ছোট analytics application বানাতে পারবে।

১৪.১ JavaScript-এ Function কেন First-Class Value?

JavaScript-এ function একটি value। Variable-এ রাখা, argument হিসেবে পাঠানো এবং অন্য function থেকে return করা যায়। এই বৈশিষ্ট্য reusable logic ও event-driven architecture-এর ভিত্তি।

```javascript
const greet = name => `স্বাগতম, ${name}!`;
const apply = (fn, value) => fn(value);
console.log(apply(greet, 'রাহিম'));

function makeMultiplier(factor) {
  return value => value * factor;
}
const double = makeMultiplier(2);
console.log(double(12)); // 24
```

`makeMultiplier` একটি নতুন function return করে। `factor`-এর মান return-এর পরেও সংরক্ষিত থাকে—এটাই closure-এর মূল ধারণা।

১৪.২ Lexical Scope ও Closure

Lexical scope অর্থ function কোথায় *লেখা হয়েছে* তার ভিত্তিতে outer variable খুঁজে পাওয়া। Closure হলো function এবং তার lexical environment-এর সংযোগ।

```javascript
function createCounter(initial = 0) {
  let count = initial;
  return {
    increment(step = 1) { count += step; return count; },
    decrement(step = 1) { count -= step; return count; },
    getValue() { return count; }
  };
}
const a = createCounter(10);
const b = createCounter();
a.increment(); // 11
b.increment(); // 1 — state আলাদা
```

Closure private state-এর জন্য উপযোগী, কিন্তু এটি নিরাপত্তার পূর্ণ বিকল্প নয়। কোনো sensitive secret frontend source code-এ রাখা উচিত নয়।

Loop-এর closure trap

```javascript
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 3, 3, 3
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0); // 0, 1, 2
}
```

`var`-এর একই binding পুনঃব্যবহার হয়; `let` প্রতিটি iteration-এ নতুন binding তৈরি করে।

১৪.৩ Higher-Order Functions

যে function অন্য function গ্রহণ করে অথবা return করে তাকে higher-order function বলা হয়। `map`, `filter`, `reduce`, `some`, `every` এর পরিচিত উদাহরণ।

```javascript
const prices = [100, 250, 400];
const taxed = prices.map(price => price * 1.05);
const expensive = taxed.filter(price => price >= 200);
const total = expensive.reduce((sum, price) => sum + price, 0);
```

কখন কোনটি? `map` প্রতিটি item বদলে নতুন array, `filter` শর্তসাপেক্ষ subset, `reduce` একটি accumulated result তৈরি করে। `forEach` side effect-এর জন্য ব্যবহার করো; এটি নতুন transformed array return করে না।

১৪.৪ Callback, `this` ও Arrow Function

```javascript
const account = {
  owner: 'Mina',
  balance: 500,
  print() { return `${this.owner}: ${this.balance}`; }
};
console.log(account.print());
const detached = account.print;
// detached() strict-mode context-এ this undefined হতে পারে
const bound = account.print.bind(account);
console.log(bound());
```

Arrow function নিজস্ব `this` তৈরি করে না। Object method-এ সাধারণ method syntax ভালো default; nested callback-এ arrow function প্রাসঙ্গিক হতে পারে।

১৪.৫ Function Composition এবং Pipeline

একাধিক ছোট function একত্র করে বড় transformation তৈরি করা যায়।

```javascript
const pipe = (...fns) => input => fns.reduce((value, fn) => fn(value), input);
const trim = value => value.trim();
const lower = value => value.toLowerCase();
const slugify = value => value.replace(/\s+/g, '-');
const toSlug = pipe(trim, lower, slugify);
console.log(toSlug('  Hello JavaScript  ')); // hello-javascript
```

`pipe(f, g)(x)` মানে `g(f(x))`। বিপরীতে `compose(f, g)(x)` মানে `f(g(x))`। Production-এ প্রতিটি transformation-এর input/output contract স্পষ্ট রাখো।

১৪.৬ Currying এবং Partial Application

```javascript
const add = a => b => a + b;
const addVat = rate => amount => amount + amount * rate;
const withVat15 = addVat(0.15);
console.log(withVat15(100)); // 115
```

Currying একাধিক argument-এর function-কে এক-argument function-এর ধারায় প্রকাশ করে। Partial application কিছু argument আগেই নির্ধারণ করে। দুটোই reusable configuration তৈরিতে কাজে লাগে, তবে অকারণে nested function বাড়িয়ে readability কমানো ঠিক নয়।

১৪.৭ Pure Function, Immutability ও Side Effects

Pure function একই input-এ একই output দেয় এবং বাইরের state বদলায় না।

```javascript
const addItem = (items, item) => [...items, item];
const updatePrice = (items, id, price) =>
  items.map(item => item.id === id ? { ...item, price } : item);
```

`sort()` মূল array বদলে দেয়; `toSorted()` নতুন array return করে (পুরোনো runtime-এ নাও থাকতে পারে)। Compatible বিকল্প `[...items].sort(...)`। Spread একটি **shallow copy**; nested object স্বয়ংক্রিয়ভাবে deep-clone হয় না।

১৪.৮ Memoization: লাভ এবং সতর্কতা

```javascript
function memoizeUnary(fn) {
  const cache = new Map();
  return value => {
    if (cache.has(value)) return cache.get(value);
    const result = fn(value);
    cache.set(value, result);
    return result;
  };
}
const square = memoizeUnary(n => n * n);
```

Memoization pure এবং costly computation-এ সহায়ক। কিন্তু unbounded `Map` memory বাড়াতে পারে। Object keys-এর ক্ষেত্রে identity-ভিত্তিক caching, mutation এবং stale result-এর সমস্যা বিবেচনা করো।

১৪.৯ Debounce ও Throttle

Search input-এ প্রতি keystroke-এ network request না পাঠিয়ে debounce ব্যবহার করা হয়। Scroll বা resize-এর frequent handler-এ throttle কার্যকর হতে পারে।

```javascript
function debounce(fn, delay = 250) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

এই সাধারণ implementation-এ `this` preserve হয় না এবং `cancel()` নেই; production utility-তে প্রয়োজন অনুযায়ী দুটোই যোগ করো।

১৪.১০ Practical Project — Sales Function Lab

এখানে functional patterns দিয়ে sales records filter, normalize, aggregate এবং rank করা হবে। Source files: `source-code/chapter-14/`।

Data Contract

প্রতিটি sale: `{ id, category, quantity, unitPrice }`। `quantity` positive integer এবং `unitPrice` non-negative finite number হতে হবে। Total = quantity × unitPrice।

Functional Pipeline

```javascript
const categorySummary = pipe(
  records => records.filter(row => row.category === 'Books'),
  records => records.map(row => ({ ...row, total: row.quantity * row.unitPrice })),
  records => records.reduce((sum, row) => sum + row.total, 0)
);
```

Actual project-এ shared validation, category aggregation, ranking ও HTML table rendering রয়েছে। User input DOM-এ `textContent` দিয়ে render করা হয়; raw HTML injection নয়।

Run Instructions

1. `source-code/chapter-14/` folder খোলো।
2. `index.html` browser-এ open করো। Sample sales load, category filter এবং summary পরীক্ষা করো।
3. Terminal-এ `node --test source-code/chapter-14/analytics.test.mjs` চালাও (Node.js-এর সাম্প্রতিক version প্রয়োজন)।

১৪.১১ সাধারণ ভুল ও Debugging Checklist

- `map`-এ return ভুলে গেলে result-এ `undefined` আসতে পারে।
- `reduce`-এর initial value না দিলে empty array-এ error হতে পারে।
- `sort` mutate করে; original data রাখতে copy করো।
- Closure-এর private state-কে shared global state ভেবে ভুল করো না।
- Floating-point অর্থের হিসাবের ক্ষেত্রে currency minor units (যেমন পয়সা) অথবা decimal library বিবেচনা করো।
- Function-এর মধ্যে DOM, storage, network এবং business calculation গুলিয়ে ফেলো না।

১৪.১২ অনুশীলন

1. `createCounter(5)` দিয়ে দুটি independent counter তৈরি করো।
2. `pipe` লিখে trim → lowercase → whitespace replace করো।
3. `filter` + `reduce` দিয়ে 1000-এর বেশি মোট বিক্রয় বের করো।
4. `makeDiscount(rate)` নামের curried function তৈরি করো।
5. Mutation ছাড়া একটি object-এর nested `address.city` update করো।
6. `memoizeUnary`-তে `cache.has` কেন `cache.get(...) || ...` থেকে নিরাপদ?
7. `debounce` ও `throttle`-এর ব্যবহারিক পার্থক্য লিখো।

8. Sales Function Lab-এ category অনুযায়ী highest revenue highlight করো।
9. Invalid quantity ও negative price-এর জন্য unit test লেখো।
10. `map`, `filter`, `reduce`-এর প্রত্যেকটির জন্য একটি real-world উদাহরণ লেখো।

১৪.১৩ অনুশীলনের নমুনা সমাধান

```javascript
// 1
const first = createCounter(5), second = createCounter(5);
first.increment(); // 6; second.getValue() এখনও 5

// 2
const slug = pipe(s => s.trim(), s => s.toLowerCase(), s => s.replace(/\s+/g, '-'));

// 3
const bigSales = sales => sales.filter(s => s.quantity * s.unitPrice > 1000)
  .reduce((sum, s) => sum + s.quantity * s.unitPrice, 0);

// 4
const makeDiscount = rate => amount => amount * (1 - rate);

// 5
const changeCity = (person, city) => ({
  ...person, address: { ...person.address, city }
});

// 6: Cached result 0, false, '' বা undefined হলেও has() key presence যাচাই করে।
// 7: debounce শেষ event-এর পরে অপেক্ষা করে; throttle সময়সীমায় call rate সীমিত করে।
// 8: Object.entries(summary).sort((a,b) => b[1]-a[1])[0] থেকে top category।
// 9: Test file-এ invalid input-এর assert.throws() আছে।
// 10: map = price transform, filter = in-stock products, reduce = cart total।
```

১৪.১৪ Knowledge Check — MCQ

1. Closure কোনটি ধরে রাখে? A. HTML B. Lexical environment C. CSS D. Browser tab
2. `map()` কী return করে? A. নতুন array B. Boolean C. DOM D. কিছুই না
3. `filter()`-এর কাজ কী? A. sort B. subset C. mutation বাধ্যতামূলক D. JSON parse
4. `reduce()`-এ initial value কেন ভালো? A. CSS B. empty input safety C. network D. hoisting
5. Arrow function-এর নিজস্ব কোনটি নেই? A. lexical `this` binding B. body C. return D. parameter
6. Pure function কী করে না? A. calculation B. return C. external state mutation D. parameters গ্রহণ
7. `pipe(f,g)(x)` কী? A. f(g(x)) B. g(f(x)) C. x D. undefined
8. `sort()`-এর আচরণ? A. original array mutate B. always immutable C. Promise D. closure
9. Memoization-এর ঝুঁকি? A. memory growth B. HTML parsing C. CSS conflict D. syntax always fails
10. Search typing-এ request কমাতে কোনটি সাধারণত উপযোগী? A. debounce B. recursion C. prototype D. `var`

Answer Key: 1-B (lexical variables), 2-A (mapped array), 3-B (matching subset), 4-B (empty array-এ safe accumulation), 5-A (arrow lexical `this` ব্যবহার করে), 6-C (side-effect এড়িয়ে চলে), 7-B (left-to-right pipeline), 8-A (in-place sort), 9-A (unbounded cache), 10-A (typing থামার পরে action)।

১৪.১৫ Chapter Summary ও Next Steps

Function-কে value হিসেবে ব্যবহার, closure দিয়ে private state, higher-order transformation, composition, immutability এবং memoization-এর সীমাবদ্ধতা—এগুলো বড় JavaScript application-এ modular design-এর ভিত্তি। পরের অধ্যায়ে Browser APIs, Storage, Fetch এবং বাস্তব Web Application তৈরি করা হবে।